Agent = Harness(Model): Five Mechanistic Primitives, and What Is Not One

Anonymous authors
Paper under double-blind review

Abstract

The software around a language model in an agent is called a scaffold, a framework, a runtime or a harness, and none of these names comes with a definition of what the thing contains. We take the position that an LLM agent is a model wrapped in a harness, *Agent* = *Harness*(*Model*), and that every harness is built from five mechanistic primitives: Input, which captures what comes in; Context, which assembles what the model is shown; Model interface, which sends the request and brings back the response; Output, which carries the response out; and Control flow, which decides whether to go again, hand back or stop. Each primitive is defined by what it does and by what it never does, so that the five are mutually exclusive. We then separate three levels at which harnesses can be described: the *primitives*, which every harness has; the *design decisions* made within each primitive, which is where harnesses differ; and the *production concerns* (persistence, safety and security, observability, resilience, performance, evaluation), which are questions asked of every primitive rather than parts of the harness. As evidence, we deconstruct six harnesses from their primary sources at pinned versions: Claude Code, Codex CLI, opencode, pi, Hermes Agent, and a LangGraph workflow. Every component we found has a home in one primitive or one concern, and the differences between the six are design decisions within the primitives. The vocabulary gives a procedure for locating an agent's failure in one of five places before attributing it to the model, and a schema for reporting the harness alongside the model. The minimal harnesses from which the definitions were drawn are reproduced in an appendix, and the evidence for every assignment is published with the paper.

1 Introduction

A language model does one thing: given a sequence of tokens, it produces a distribution over the next token. It cannot see a file it was not shown, remember a conversation it was not sent again, or carry out a command it wrote. Language models are nonetheless described as agents that read codebases, run tests, browse the web and fix bugs. The difference between what a model is and what an agent does is software around the model. That software is the subject of this paper.

The software has many names. Evaluations call it a *scaffold* (Anthropic, 2025; METR, 2025); libraries call themselves *frameworks*, *SDKs* or an *orchestration framework and runtime* (LangChain, 2026); and *harness* is now common among people who build coding agents (Young, 2025). None of the names says what the thing contains, where it ends and the model begins, or how two instances of it differ. As a result, agents are compared by the model inside them, and results on one benchmark with one model vary with the system around it (§6). When such an agent fails, there is no agreed list of places to look.

Position. We argue three claims.

1. **An LLM agent is a model wrapped in a harness**: *Agent* = *Harness*(*Model*). The model maps tokens to a next token. The harness is everything else on the path of a request.

2. **Every harness is built from five mechanistic primitives**: INPUT, CONTEXT, MODEL INTERFACE, OUTPUT and CONTROL FLOW. Each is defined by what it does and by what it never does, so that each piece of a harness has one home.
3. **Everything else said about a harness is at one of two other levels**: a *design decision* within a primitive (one tool or forty; a loop or a graph; a repository map or a summarizer), or a *production concern* asked of every primitive (persistence, safety and security, observability, resilience, performance, evaluation). Neither level adds a primitive.

What the vocabulary is for. It separates what a harness *is* from what a given harness *chose* and from what makes it fit to *run*. With that separation, a failure can be located: was the model shown what it needed (CONTEXT), did what it asked for happen (OUTPUT), did the loop stop early or run long (CONTROL FLOW), was the response read correctly (MODEL INTERFACE), did the right thing come in (INPUT)? Only when all five are ruled out does the failure belong to the model. Two harnesses can be compared decision by decision within the same five primitives, a benchmark result can report the harness that produced it, and a harness can be moved to a new model by changing one primitive.

Evidence. The definitions come from building a harness from nothing, one primitive at a time, around a fixed model and a fixed task, and observing what each version could not do (Appendix A). The claim that the five cover existing harnesses is tested by deconstructing six of them from their primary sources at pinned versions: Claude Code, Codex CLI, opencode, pi, Hermes Agent, and a LangGraph workflow (§5). For each we assign every component on the request path to one primitive, or to one concern together with the primitive it changes, and record every case that was hard to place. The evidence for each assignment is given in Appendix B and published with the paper.

What this paper is not. It is a conceptual contribution. It reports no experiment in which a primitive is varied and performance measured, and it does not prove that no sixth primitive could ever be needed. It states what would refute the claim (§8) and gives a deconstruction a reader can check.

Contributions.

- A definition of the harness and of its boundary with the model (§2).
- Five primitives, each with a mechanism, a negative definition, and its design space (§3).
- Three levels of description, primitive, design decision and production concern, with rules for the cases at their boundaries (§4).
- A deconstruction of six harnesses, with method, selection criteria and published evidence (§5, Appendix B).
- Uses of the vocabulary: a diagnostic procedure, a reporting schema, and porting (§6).
- The minimal harnesses, one primitive at a time (Appendix A).

2 The Boundary: Model and Harness

The model. We take a language model to be a function from a token sequence to a distribution over the next token, applied autoregressively until a stop token is produced:

$$TokensOut = Model(TokensIn).$$

Chat templates, tool-call formats and reasoning tokens are conventions for laying out *TokensIn* and reading *TokensOut*. Models learn these conventions in training, and can learn to emit calls to tools (Schick et al., 2023), but the calls are carried out, and the conventions written and parsed, by something else. Viewed from outside, the model keeps no state between calls, cannot read from or act on its environment, and cannot call itself again.

The harness. The harness is everything in an agent that is not the model: the software that turns the world into tokens for the model, and the model's tokens into effects on the world.

$$Agent = Harness(Model).$$

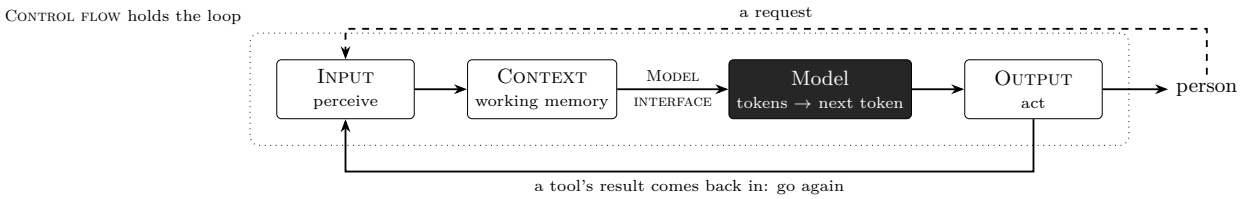


Figure 1: The agent loop. The five primitives surround the model; Control flow decides whether the loop runs again, hands back, or stops.

Algorithm 1 A harness. Lines are labelled with the primitive they implement.

```
 1: procedure ControlFlow( )
 2:     conversation ← [ ]
 3:     loop                                          ▷ outer loop: one request at a time
 4:         conversation.append(User(Input( )))        ▷ Input: from a person
 5:         loop                                      ▷ inner loop: one request, carried through
 6:             ctx ← Context(conversation)           ▷ Context
 7:             response ← ModelInterface(ctx)        ▷ Model interface
 8:             conversation.append(Assistant(response))
 9:             results ← Output(response)            ▷ Output: show words, run actions
10:             if results = ∅ then break             ▷ nothing ran: hand back
11:             conversation.append(User(results))    ▷ Input: from the world
```

The boundary is at the model's weights and its sampling. Everything the model is shown, everything it can do, and everything that happens between one call and the next is on the harness side. This includes things often attributed to the model: memory across sessions (the harness stores and retrieves it), tool use (the model emits a request; the harness carries it out), and knowledge of the date or the working directory (the harness writes it into the context).

The loop. Figure 1 shows the shape every harness shares. Something comes in: a request from a person, an event, or the result of an earlier action. The harness assembles a context around it and sends the context to the model. The model runs its own inner loop, one token at a time, until it stops. The harness handles the response: it shows words to a person, or carries out the actions the response requests. If an action produced a result, the result comes back in and the loop runs again; if not, control returns to the person, or the agent stops.

Scope. The primitives describe what happens on the path of a request at run time. Software that ships with a harness but never acts on that path is out of scope: hosting and transport, configuration loading, packaging, the user-interface framework, and the infrastructure that carries telemetry (what is recorded is in scope; the pipe that carries it is not). Model development is out of scope too: the model is held fixed.

Not every harness is an agent's. A program that makes one model call and prints the reply has a harness, but a trivial one: its input is fixed, its context is one message, its output is printed, and its control flow is a straight line. The primitives describe this case as well. What makes a harness an agent's is a Control flow that feeds the results of actions back to the model.

3 The Five Primitives

Algorithm 1 gives a harness as pseudocode, each line labelled with its primitive. Every harness in §5 is an elaboration of it. For each primitive we give its *mechanism* (what it does on the path of a request), its *negative definition* (what it never does, and which primitive does), and its *design space* (the decisions a harness makes within it). The negative definitions make the five mutually exclusive: each piece of a harness has one home, assigned by its function rather than by its name or its location in the code.

3.1 INPUT: capture what comes in

Mechanism. Capture what enters the agent and hand it to CONTEXT. There are two sources: a person (a message, a file, an interrupt, an answer to a question the model asked) and the world (the result of an action, an event, a scheduled trigger). **Never.** INPUT never decides what the model is shown; that is CONTEXT. It never runs anything; that is OUTPUT. An answer a person gives that the *harness* consumes, such as an approval, is not input but a CONTROL FLOW decision; an answer the *model* consumes, returned as a tool result, is input. **Design space.** The source: a terminal, an editor, a chat channel, a command-line argument, an issue tracker, a webhook, a message queue. The kind: text, images, files, audio, the results of actions, interrupts, and mid-turn steering. The second source, results from the world, is what makes acting worthwhile: an action's result is new perception.

3.2 CONTEXT: assemble what the model is shown

Mechanism. Assemble, for one call, everything the model is shown: instructions, the tools' descriptions, the conversation so far, and anything retrieved from memory or the environment; and fit it to the window, by truncation, compaction or selection. CONTEXT is the agent's working memory, rebuilt on every call. **Never.** CONTEXT never calls the model; that is MODEL INTERFACE. It never acts on the world; that is OUTPUT. Reading a memory store is CONTEXT; writing one, if the model requests it, is an action and so OUTPUT. Code that decides what the model sees is CONTEXT even when it sits inside a tool: a tool that attaches project instructions to a file it reads, or condenses a web page before returning it, is doing CONTEXT's work. **Design space.** What goes in the instructions (identity, rules, the agent's own capabilities, project files such as `AGENTS.md`); how tools are described; how long-term memory comes in (preloaded, retrieved on demand, or searched by the agent through its own tools); the roles each piece is placed in, which carry authority; and what happens when the window fills: drop old turns, summarize them, or include only what the moment needs.

3.3 MODEL INTERFACE: reach the model

Mechanism. Send the assembled context to the model and bring back its response in a form the rest of the harness can read: words, and structured requests to act. This includes the client, the wire format, the translation between the harness's message format and the model's chat template, streaming, retrying a failed request, and switching to a backup model when the request could not be served. **Never.** MODEL INTERFACE never decides what the request contains; that is CONTEXT. It never acts on the response; that is OUTPUT. It receives a stream; rendering each fragment as it arrives is OUTPUT. A change of model because the request could not be served (unavailable, overloaded, refused) is MODEL INTERFACE; a change of model keyed to the work (planning, a cheap model for a side task) is CONTROL FLOW. **Design space.** A hosted API or local inference; one provider or many, directly or through a translation layer; native tool-call formats or tool calls parsed from text; streaming or whole responses; retry policy and backup models. MODEL INTERFACE is the only primitive that knows which model is inside, which is why changing the model means changing it alone.

3.4 OUTPUT: carry the response out

Mechanism. Handle what the model returned: show its words to a person, and carry out each requested action, collecting its result. The tool set, the executor that runs an action, and the environment it runs in are OUTPUT. **Never.** OUTPUT never decides whether to go again; that is CONTROL FLOW. It never assembles the next request; that is CONTEXT. The model only ever *requests* an action; OUTPUT decides whether and how it happens, which is where sandboxes and never-lists live. Several tool calls from one response run concurrently: OUTPUT. Several *model* calls run concurrently: CONTROL FLOW. A program the model writes at run time that calls tools without calling the model is OUTPUT; one that sequences model calls is CONTROL FLOW. **Design space.** The tool set, from one general tool (a shell) to dozens of specific ones (read, write, edit, search, browse, computer use); the action format, from structured tool calls to code the model writes and the harness executes (Wang et al., 2024) to edit formats such as search-and-replace blocks; the execution

environment, from the host machine to a container or a remote sandbox; and the presentation of results, including truncation and error messages, which Yang et al. (2024) show affect performance.

3.5 Control flow: decide what happens next

Mechanism. Define the shape of the loop: send results back and go again, hand back to the person, or stop. Step and token budgets, approval gates the harness consumes, delegation to sub-agents and the sequencing of several agents, and routing to a model keyed to the work are all Control flow. **Never.** Control flow never captures input, assembles context, calls the model or runs a tool. It sequences the other four. A workflow in which code fixes the order of model calls (Schluntz & Zhang, 2024) and an agent in which the model directs its own process differ here and only here. **Design space.** A single open loop that runs until the model stops requesting actions; a fixed pipeline; an explicit graph; sub-agents with contexts of their own; planning or approval modes that change what is allowed; and stopping rules: step limits, budgets, interrupts, loop detection.

3.6 Necessity

Each primitive answers a question no other answers: what comes in, what the model is shown, how the model is reached, what is done with what comes out, and what happens next. Appendix A gives the five minimal harnesses from which the definitions were drawn, built one primitive at a time around a fixed model and task. Each version fails in a specific way without the next primitive: with only Model interface, the model answers but cannot act and nobody can ask it anything else; with Input, anyone can ask and it still cannot act; with Output, it acts once and the result goes nowhere; with Control flow, it finishes the task but does not know what it is or where it is, and spends actions finding out; with Context, it knows. This shows that each primitive removes a failure the others do not. It does not show that no sixth primitive could ever be needed; §5 tests that against existing harnesses, and §8 says what would refute it.

4 Three Levels: Primitive, Design Decision, Production Concern

Descriptions of harnesses mix three kinds of statement. "It has a loop" names a primitive. "It gives the model forty tools" names a decision within one. "It runs commands in a sandbox" names a measure taken for a reason that applies to every primitive. Keeping the three apart is the main use of the vocabulary, and the deconstruction in §5 records each component at its level.

Primitives. A primitive is a mechanism every harness has, defined by its function on the request path (§3). A harness has exactly five. A harness that lacks one is not an agent's harness, or is not a harness.

Design decisions. A design decision is one way of implementing a primitive. It is the level at which harnesses differ, and the level at which they are usually described: a single shell tool or a set of file tools (Output); a repository map or a project file in the instructions (Context); a loop that runs until the model stops or a graph with named nodes (Control flow); one provider or a gateway to many (Model interface); a terminal or a message queue (Input). A design decision never adds a primitive; it fills one. The test for whether something is a design decision is whether a different harness could make a different choice and still be a harness.

Production concerns. A production concern is a question asked of every primitive, whose answer changes how some primitives are implemented. We use six (Table 1): what should outlast the session (persistence); what could do harm, and who could make it (safety and security); what did it do (observability); what if something breaks (resilience); what is slow or costly (performance); and how do we know it still works (evaluation). A concern is not a part of the harness. A sandbox is not a sixth primitive; it is the safety answer in Output. A memory store is not a sixth primitive; it is the persistence answer in Context and Output. The test for whether something is a concern is whether it is motivated by a question that could equally be asked of a different primitive.

Table 1: Six production concerns as questions asked of each primitive. Each cell names what the concern changes in that primitive; "—" means we found nothing essential there in the harnesses examined.

	Persistence *What outlasts the session?*	**Safety & security** *What could do harm, and who could make it?*	**Observability** *What did it do?*	**Resilience** *What if something breaks?*	**Performance** *What is slow or costly?*	**Evaluation** *Does it still work?*
Input	an unfinished session offered back	approvals asked of a person	refusals recorded	input ending; interrupts	—	yes to every approval in a test run
Context	memories; resumed transcripts	untrusted content; poisoned memory; roles and authority	—	compaction when too long	caching the unchanged prefix; trimming results	the instructions under test
Model interface	—	—	tokens, latency, cost per call	retries; a backup model	streaming	the model under test
Output	—	sandboxes; never-lists; secrets hidden; pinned images	what each action did	time limits on actions	parallel actions; smaller models for side tasks	the tools under test
Control flow	the task in progress; a diary of every message	runaway loops; the record kept out of reach	each decision	resume after a crash	—	the loop under test

The concerns interact. Persistence creates a security surface: a poisoned memory steers every later session. Streaming changes safety: an interrupt can stop a reply mid-sentence, and the harness must record what was said rather than a finished answer. Summarizing changes persistence: the summary must say where the originals are kept. Treating concerns as questions asked of each primitive makes these interactions visible; treating them as separate components hides them.

4.1 Rules for the cases at the boundaries

The negative definitions decide most cases. The following rules decide the rest. Each was needed for a component found in §5.

Function, not location.
A component is assigned by what it does, not by the file or the name it has. Code inside a tool that decides what the model sees is Context. An approval decision made inside a tool's runtime is Control flow.

The tool channel.
The model drives other primitives through tool calls: spawning a sub-agent, entering a planning mode, writing a plan, searching for tools, asking the person a question. The call is Output; its effect is assigned to the primitive it changes.

Backup versus routing.
A model change triggered by a request that could not be served is Model interface. A model change keyed to the work is Control flow. A fixed model for one class of request, such as compaction summaries, is a request setting and Model interface.

Receiving versus rendering.
Receiving a response as a stream is Model interface; rendering each fragment is Output.

Parallel actions versus parallel calls.
Several tool calls from one response run concurrently: Output. Several model calls issued concurrently, to divide a task or to vote: Control flow.

The reader of a record.
A record kept for the model (a searchable transcript) is Context. A record kept for the operator (a trace) is observability. An artifact for the user (a diff view) is Output. One file may serve several readers, and each use is assigned separately.

The consumer of an answer.
An answer the harness consumes (an approval) is Control flow. An answer the model consumes (the result of an ask-the-user tool) is Input.

Direct actions by a person.
A command the person runs through the harness's own executor, whose output enters the conversation, is INPUT, although it reuses OUTPUT's executor.

Model-written programs.
A program the model writes at run time is CONTROL FLOW if it sequences model calls and OUTPUT if it sequences only tools.

Bundles.
A mode, a skill, a hook system or an agent profile is packaging that bundles decisions across several primitives. Decomposed, each element has one home: a planning mode is a context template, a restricted tool set and an approval gate; a hook system is a dispatcher (CONTROL FLOW) whose events each affect one primitive.

Nested harnesses.
A sub-agent is a harness of its own, and delegation to it is CONTROL FLOW of the outer harness. The same holds for model-backed components inside a harness, such as an approval reviewer or a termination judge: each is CONTROL FLOW with a MODEL INTERFACE call.

Primitives as services.
One primitive may use another's mechanism to do its job: CONTEXT asks the model to summarize old turns before compacting them; a web-fetch tool in OUTPUT asks the model to condense a page before returning it. The component is assigned by what it *decides* (what to compact, what to return), and the call it makes is a use of MODEL INTERFACE. The negative definitions are about responsibility, not about which function calls which.

Tool protocols.
A protocol such as MCP (Model Context Protocol, 2026) provides pluggable OUTPUT (third-party tools). Its other features are assigned by function: resources and prompts are CONTEXT; sampling borrows the client's MODEL INTERFACE; elicitation is INPUT.

5 Deconstruction of Six Harnesses

If the position holds, a harness built by someone else, for a different purpose, at a different scale, should decompose into the five primitives with nothing left over, and its differences from other harnesses should be locatable as design decisions within a primitive. We test this on six.

Selection. We chose harnesses in wide use whose primary sources were available, across three kinds: two command-line coding agents from model developers (Claude Code (Anthropic, 2026) and Codex CLI (OpenAI, 2026)), two open-source coding agents built outside the model developers (opencode (Anomaly and contributors, 2026) and pi (Zechner, 2026)), one general-purpose agent built for messaging platforms as well as the terminal (Hermes Agent (Nous Research, 2026)), and one workflow framework (LangGraph (LangChain, 2026)), for which we deconstruct the canonical agent it ships, a ReAct loop, together with the runtime that executes it. The workflow framework is included to test the claim that a workflow and an agent differ only in CONTROL FLOW. Five of the six publish their source; Claude Code does not, and is deconstructed from its documentation. The six are not a sample of all harnesses, and §8 says what that leaves untested.

Method. For each harness we (i) read the source at a pinned commit, or the documentation on a stated date, and mapped the path of a request from the moment a message arrives to the next model call; (ii) assigned every component on that path to exactly one primitive, or to one production concern together with the primitive it changes, by the definitions of §3 and the rules of §4.1, citing a file and line range or a page and section for each; and (iii) recorded every component that was hard to place and how it was resolved, or that it was not. Components outside the request path (hosting, packaging, UI frameworks) were noted and left unassigned. The evidence is in Appendix B and in the repository as a file per harness; the subsections below summarize it.

5.1 Claude Code

Anthropic's coding harness, deconstructed from its documentation as read on 15 October 2026 (Anthropic, 2026); its source is not published. The loop is the open one of Algorithm 1: the model is called until it returns a response with no tool calls, and a turn's result carries why it ended (done, a turn limit, a cost budget, an error). INPUT has one entry point for everything that starts a turn: typed prompts, piped input, messages queued mid-turn and delivered after the current tool batch, scheduled prompts, and reports from background sub-agents. CONTEXT is built as a stable cached prefix (the system prompt, the tools, then project files such as `CLAUDE.md`) with later updates appended as messages or system reminders; tool schemas for MCP servers are deferred behind a search tool; and compaction, on reaching the window, is a separate request that re-injects the project files, memory, recently read files and invoked skills. MODEL INTERFACE is the Messages API with streaming, up to ten retries that never re-issue after a completed tool block, idle watchdogs, and a fallback chain the user configures. OUTPUT is a fixed built-in tool set plus MCP tools; read-only calls run concurrently and mutating calls in sequence; shell commands run under an OS sandbox with a domain-allowlist proxy; large results spill to files; file edits are checkpointed for rewind. CONTROL FLOW adds a six-step permission pipeline (hooks, deny rules, ask rules, the permission mode, allow rules, then a prompt), delegation through an agent tool to a depth of three, and, at scale, a model-written JavaScript workflow the runtime executes, which sequences model calls and is therefore CONTROL FLOW by the rule on model-written programs.

Three components were hard to place. The web-fetch tool calls the model itself to condense a page before the main model sees it: the call is OUTPUT, the condensing is a CONTEXT decision, and the extra request is MODEL INTERFACE. The default permission gate in "auto" mode is a second model asked to classify each action: a CONTROL FLOW gate implemented as a model call with its own stripped context. And a server-side variant of that classifier runs inside the API during the session's own requests, so the harness only consumes its verdict; we return to this in §5.7.

5.2 Codex CLI

OpenAI's terminal and IDE coding agent, at commit `f0c5077` (OpenAI, 2026), written in Rust. A session is one thread that receives typed operations over a channel; a dispatcher matches each one (a turn's input, an interrupt, an approval, the answer to a question, a compaction request) and runs each user turn as a task. INPUT is that channel: text typed during a running turn is queued as steering rather than starting a new turn, an inter-agent mailbox and asynchronous hook results merge into the same pending-input queue, and the model's questions to the person come back as a typed answer. CONTEXT is the Responses API's own item list, with the environment and instructions injected as typed "world-state" sections sent as diffs when they change, project files walked from the root to the working directory, and tool outputs truncated in the live history only; compaction has three implementations (a local summary, a remote one performed by the server, and a token-budget reset), chosen by explicit threshold arithmetic, and can run speculatively in the background once usage passes a percentage of the window. MODEL INTERFACE is the Responses API only, WebSocket first with a session-wide fallback to HTTP, five retries with jittered backoff that honour the server's retry timing, an idle timeout, and requests sent without server-side storage, with encrypted reasoning replayed. OUTPUT routes every risky action through one orchestrator (approval, then the platform sandbox, then the run, then a retry if the sandbox denied it), with Seatbelt on macOS, seccomp with bubblewrap and Landlock on Linux and a restricted token on Windows; tools run concurrently under a read–write gate that lets parallel-safe tools overlap; and a "code mode" runs model-written JavaScript in V8, each nested tool call re-entering the harness's approval and sandbox path. CONTROL FLOW is one task per turn, looping until the model emits no tool call, with stop hooks, roll-over into compaction mid-turn, an approval precedence of hooks, then an optional model reviewer ("Guardian"), then the person, and sub-agents that are full sessions of their own with depth and thread caps.

Hard cases: the Guardian is a CONTROL FLOW gate whose verdict comes from a nested harness; the remote compaction is a CONTEXT decision whose summarizer is the provider; over WebSocket the request omits the tools and instructions the server already holds, so what is sent depends on transport state while what the model is shown does not; and the plan-mode parser, which reads the streamed text into a proposed-plan item

and can redirect it, is Output rendering that also feeds the stop decision. We record the last as straddling Output and Control flow by design.

5.3 opencode

An open-source coding harness by Anomaly, at commit `654b79b` (Anomaly and contributors, 2026); it was rewritten since earlier versions, and the deconstruction describes the current one. Each session is an event-sourced record in SQLite, and one fibre per session drains a durable inbox: Input is a row in that inbox, from a person, a shell command, a skill, or the harness itself, marked for delivery either at the next step boundary ("steer") or after the current turn ("queue"). Context re-projects the event history into model messages on every call, with instructions placed in a cached prefix and changes to them frozen into the history as dated events; compaction is triggered at about ninety percent of the window, using provider-measured token counts plus a local estimate, keeps the newest fifteen thousand tokens verbatim, and can be delegated to the provider's native compaction. Model interface is an in-house request type compiled per provider protocol (Anthropic Messages, OpenAI Responses, Bedrock Converse, Gemini), streamed over HTTP or a stateful WebSocket, with a retry policy split by whether output had already started. Output runs tool calls concurrently as they stream, and most tools are reachable only through one `execute` tool that runs model-written JavaScript in an interpreter that cannot call the model, which the rule on model-written programs places in Output. Control flow continues while a step made tool calls, stops on a text-only finish, enforces an optional per-agent step budget, and delegates to child sessions (depth one by default), whose results return as tool text or as a synthetic message.

Hard cases: every tool asserts its own permission, so an approval gate the harness consumes lives inside the tool's code (Control flow by function); the step budget is enforced by appending an assistant message and disabling tools, a Control flow decision carried out by editing what the model sees; and the compaction service both decides what to summarize and issues the model call itself. Native provider-side compaction leaves the harness's Context holding an item it cannot inspect.

5.4 pi

A terminal coding agent by Mario Zechner, at commit `5ab7fe7` (Zechner, 2026), built in layers: a minimal agent loop, a provider layer, and a coding-agent session on top. Input comes from a terminal editor, a print mode, or a JSON-RPC mode, all into one `prompt()` call, with mid-run messages split into steering (after this turn) and follow-ups (after the agent would have stopped). Context is a projection of a persisted session tree: a compaction entry, the kept tail, and context edits, with the system prompt kept as named sections and diffed into patches, so that replaying the file reproduces the exact request; compaction triggers when the context exceeds the window less a reserve, and keeps the most recent twenty thousand tokens. Model interface is one streaming function behind about fifteen provider adapters, with the SDKs' own retries disabled so that the harness owns a two-tier retry policy, and an overflow classifier that routes a too-long request to compaction rather than to a retry. Output is a set of plain tool objects run in parallel by default, with mutations to one file serialized; there is no sandbox in the core (OS sandboxing and permission prompts are extensions), and a "code mode" tool runs model-written scripts in a QuickJS sandbox. Control flow is a two-level loop (tool calls and steering inside, follow-ups outside) plus a session-level loop for retries, overflow recovery and extension continuations; sub-agents are not in the core, though an example spawns child processes; and per-request routing between "virtual models" is keyed to the reason for the request.

Hard cases: a wire-level hook lets an extension rewrite the request body, which is Context by function though it sits in the adapter; the virtual-model router has a "retry" branch that overlaps the backup-model rule (we place it in Control flow, since the router decides by the state of the work); and the code-mode script can call auxiliary models from inside the sandbox, a model call issued from inside a tool, which we discuss in §5.7. A telemetry interface exists, but no span is emitted at this commit.

5.5 Hermes Agent

A personal agent by Nous Research, at commit `7bf3766` (Nous Research, 2026), with one agent core behind a terminal interface, about twenty-five messaging-platform adapters, an HTTP server, an ACP adapter and a cron scheduler. INPUT is one normalized message event shared by all of these; mid-turn messages become steering rather than new turns. CONTEXT is a three-tier system prompt, kept byte-stable so that caching holds: identity, then project files (capped by size), then volatile memory, skills and the time; compaction at half the window prunes, then summarizes with an auxiliary model; oversized tool output is spilled to files rather than dropped; and a tool-search bridge lets the model fetch tool schemas on demand. MODEL INTERFACE is an OpenAI-style chat-completions client by default with pluggable transports for Anthropic, Responses and Bedrock, streaming on, three retries with backoff, then a configurable fallback chain, then a bounded auto-recovery wait. OUTPUT sends a fixed core tool set on every call, splits each batch into parallel-safe segments, runs terminal commands behind pluggable sandboxes (local, Docker, Modal, and others) and a regex-plus-human approval gate, and persists each tool-call row before executing it. CONTROL FLOW is a loop of named phases, each returning a verdict, with an unlimited iteration budget by default, "stop-gate" nudges that append a synthetic message and go again, flat delegation to threaded children, and task-keyed routing to auxiliary models.

Two components do not fit cleanly. In one mode the whole turn is handed to an external Codex process, and Hermes keeps only input, persistence and a fallback: the five primitives collapse into one opaque external harness that is neither a sub-agent (there is no outer loop) nor the model. And "providers" reached over ACP make the thing behind the MODEL INTERFACE another agent. Both are nested harnesses by the rule of §4.1, but of an unusual shape: the nesting is at the MODEL INTERFACE, not in CONTROL FLOW.

5.6 A LangGraph workflow

LangGraph (LangChain, 2026), at commit `5eb60ae`, is a framework for writing control flow as a graph over a checkpointed state, executed by a runtime in bulk-synchronous supersteps. We deconstruct the agent it ships, `create_react_agent`, together with that runtime. INPUT is a write to a named channel of a typed state, merged by a user-chosen reducer, whether it is a person's message, a tool result, or a resume value after an interrupt. CONTEXT is exactly the state's message list, optionally prefixed by one system message; fitting it to the window is left entirely to a user-written hook before the model call, and the framework provides only a primitive for removing messages. MODEL INTERFACE is one call to a LangChain chat model per step, and the framework contains no client of its own, no streaming of its own, and no backup model; retries and timeouts are per-node policies, off by default. OUTPUT is a tool node that runs every tool call of a response concurrently in-process, turns results and (configurably) errors into messages, and lets a tool return a routing command. CONTROL FLOW is the graph: "go again" is a conditional edge from the agent node to the tool node and back, "stop" is an empty task set or a recursion limit, and a pause is an exception the runtime persists as a checkpoint write and resumes by re-running the node.

This is the clearest case of the three-level distinction. The framework fixes the five primitives' mechanisms and leaves most of the design decisions to user code: the edges, the stopping rule, the window-fitting hook, the approval gate around an interrupt, dynamic model selection, retry policies. A workflow and an agent built with it differ in the graph, which is CONTROL FLOW, and nowhere else. The hard cases are the step counter injected into the state (it looks like CONTEXT but no model sees it; it is a CONTROL FLOW budget), and a scheduling packet that carries a state snapshot to the tool task (CONTROL FLOW carrying CONTEXT).

5.7 Summary

Table 2 gives the six side by side, one row per primitive. Every component we found has a home. The differences between the harnesses are in the rows: what comes in and from where, what the context holds and how it is fitted, how the model is reached, which tools exist and where they run, and what shape the loop has. None of the six required a sixth row.

What was hard to place. The hard cases are collected in Appendix B, and four recur across harnesses.

Table 2: Six harnesses by primitive. Each cell is the design decision that harness made within the primitive; the evidence for each is in Appendix B.

	Claude Code	Codex CLI	opencode	pi	Hermes Agent	LangGraph
Input	prompts, stdin, queued mid-turn messages, scheduled prompts, sub-agent reports, tool results, answers to an ask-user tool	one operation channel per thread; steering queued mid-turn; mailbox; typed answers to the model's questions; tool results	durable inbox rows (person, shell, skill, synthetic), delivered as steer or queue at step boundaries	terminal, print and RPC modes into one prompt call; steer and follow-up queues	one message event from CLI, 25 messaging gateways, HTTP, ACP, cron; mid-turn steering	writes to a typed state channel, merged by a reducer: messages, tool results, resume values
Context	cached prefix (system, tools, CLAUDE.md) plus appended reminders; deferred MCP schemas; compaction re-injects memory and recent files	the Responses item list; world-state sections as diffs; project files walked to cwd; three compaction modes, with background compaction	event-sourced history re-projected per call; cached instructions as dated events; compaction at ∼90%, 15k tail, or provider-native	projection of a session tree; sectioned prompt diffed into patches; compaction past a reserve, 20k tail	three-tier byte-stable prompt; memory and project files; compaction at 50% by prune then summarize; tool-search bridge	the state's messages plus one system message; window-fitting left to a user hook
Model interface	Messages API, streaming; 10 retries; idle watchdogs; user-configured fallback chain	Responses API only; WebSocket, falling back to HTTP; 5 retries with server-advised timing; no server-side storage	in-house request compiled per protocol; SSE or WebSocket; retry split by whether output started	one stream function over ∼15 adapters; SDK retries off; two-tier retry; overflow routed to compaction	chat-completions client with pluggable transports; streaming; 3 retries, fallback chain, auto-recovery	one LangChain chat-model call per step; no client, streaming or fallback of its own
Output	built-in tools plus MCP; read-only calls concurrent; OS sandbox with domain allowlist; results spill to files; edit checkpoints	one orchestrator: approve, sandbox, run, retry on denial; OS sandboxes on three platforms; read–write gate for parallel tools; V8 code mode	tools run concurrently as they stream; each asserts its own permission; most reached through a code-mode interpreter	plain tool objects, parallel by default; no core sandbox; QuickJS code mode	core tools plus bridge; parallel-safe segments; sandboxed terminal (local, Docker, Modal, . . .); approval gate	tool node runs calls concurrently in-process; errors become messages; tools may return routing commands
Control flow	loop until no tool calls; six-step permission gate, with a classifier model by default; agent tool to depth 3; model-written workflows	task per turn; loop until no tool call; hooks, Guardian reviewer, then person; sub-agents as full sessions with depth caps	one fibre per session; continue while tools called; per-agent step budget; child sessions, depth 1	inner tool loop, outer follow-up loop, session loop for retries; virtual-model routing	phase-verdict loop; unlimited budget; stop-gate nudges; flat threaded delegation; task-keyed routing	a user-written graph run as supersteps; stop on empty task set or recursion limit; interrupts checkpointed

- **Primitives that call the model.** In Claude Code, opencode and pi, compaction decides what to summarize and then calls the model to summarize it; Claude Code's web-fetch tool and pi's code mode call models from inside a tool. Read as a call graph, these break the rule that Context and Output never call the model. Read as responsibility, they do not: the component still decides one thing, and uses Model interface to do it. We added the rule on primitives as services (§4.1) for these cases.
- **Approval gates inside tools.** In opencode and Hermes the code that asks the person sits inside the tool that would act, and in Codex the approval requirement depends on what the sandbox allows. By function the gate is Control flow and the enforcement is Output; by location they are one file. The two also read each other, which is the interaction of concerns noted in §4.
- **Decisions carried out by editing the context.** opencode enforces a step budget by appending an assistant message and disabling tools; Hermes's stop gates append a synthetic message and go again; LangGraph replaces the model's answer with a canned message when the step budget is reached. Each is a Control flow decision whose mechanism is a change to what the model is shown. The decision and the mechanism have different homes, and we record both.
- **Nesting at the model interface.** Hermes can hand a whole turn to an external Codex process, or reach an agent over ACP as if it were a model. The thing behind Model interface is then another harness. The nested-harness rule covers it, but the nesting is at a different primitive from the sub-agent case, and the outer harness keeps only Input, persistence and a fallback.

One finding is not a hard case but a boundary question. Claude Code's server-side classifier runs inside the API during the session's own requests, and opencode can delegate compaction to the provider. In both, a function the vocabulary places in the harness runs on the provider's side of the API, and the local harness holds only a verdict or an opaque item. The primitives still apply: the harness is split across two machines, not reduced to four parts. But it means the boundary of §2 is at the model's weights, not at the API, and a deconstruction from a client's source alone will miss the part that runs behind the API. We found no component in the six that satisfied none of the five definitions.

6 Using the Vocabulary

6.1 Locating a failure

When an agent fails, the primitives give a short list of places to look, in order. The order matters: each question rules out a place the next question assumes.

1. Context**: was the model shown what it needed?** Read the request that was sent. If the file, the result or the instruction is not in it, the model could not have used it, and nothing downstream is at fault.
2. Output**: did what it asked for happen?** Compare the action the model requested with what ran. A parser that dropped a call, a sandbox that refused it, a tool that returned a truncated result, and an error message the model could not read are all Output.
3. Control flow**: did the loop stop early or run too long?** A step limit reached before the task was done, a hand-back on a response that still had work in it, and a loop that repeated the same action are Control flow.
4. Model interface**: was the response read correctly?** A tool call emitted in the right format but not recognized, a stream cut off and treated as complete, and a backup model substituted without notice are Model interface.
5. Input**: did the right thing come in?** A result delivered to the wrong turn, an interrupt lost, a person's answer consumed by the harness when the model needed it.
6. **Only then, the model.** If the model was shown what it needed, asked for the wrong thing, and its request was carried out as asked, the failure is the model's: its knowledge, its habits, or its size.

Two examples from our own transcripts. A small model, after instruction-tuning, sometimes emitted a second tool call as bare JSON without its `<tool_call>` tags. The harness read the JSON as words, found no action, and handed back. Question 4 locates this in Model interface, whose parser defines what counts as a call; the fix was on the model side (training on the format) but the diagnosis was the harness's. In the same experiments, four kinds of task could never pass during reinforcement learning because the check compared

the folder before and after, and the harness's own memory directory changed the folder on every run. No question in the list locates this, because the fault was in the evaluation, a concern, not in a primitive. Building the harness around a capable model first, until it passed every task in the evaluation, is what let every later failure be attributed to the model swapped in.

6.2 Reporting a harness

If *Agent* = *Harness*(*Model*), a benchmark score belongs to the pair. Holding GPT-4 Turbo fixed, Yang et al. (2024) report 1.31% of SWE-bench (Jimenez et al., 2024) resolved with retrieval-augmented generation and 12.47% with their agent, and on SWE-bench Lite 11.00% for a shell-only agent against 18.00% for SWE-agent, with only the harness changing. Anthropic notes that performance "can vary significantly based on this scaffolding, even when using the same underlying AI model" (Anthropic, 2025); METR found that tailoring a scaffold to one model more than doubled the length of tasks it could complete (METR, 2025), and later that two mature harnesses did not outperform their default scaffolds (METR, 2026), which suggests the effect is largest where the design decisions differ most. Zhang et al. (2026) argue that the harness should be disclosed whenever agents are compared, and Kapoor et al. (2025) evaluate models, scaffolds and benchmarks as separate dimensions. A disclosure needs a schema. We suggest the five primitives: the input sources; what the context holds and how it is fitted to the window; the provider, decoding settings and retry policy; the tools, their output formats and the execution environment; and the loop's shape and stopping rules. The summary table of §5 is such a disclosure for six harnesses.

6.3 Porting

Because only Model interface knows which model is inside, a harness can be moved to a new model or provider by changing that primitive alone, translating requests and responses to the new format. Our own harness was built around a hosted model and later run with a local one by replacing one function (Appendix A). A harness loses this property when model-specific details leak into other primitives: a provider's tool-call format assumed in Output, or a provider's error strings matched in Context management. The deconstruction records such leaks where we found them.

6.4 Training a model for its harness

The primitives also say what a model trained for one harness must learn: the context layout it will be shown (Context), the format in which it must request actions and the shape of the results it will read (Output), and the loop it runs in (Control flow). A model post-trained for many harnesses must guess which one it is in; a model trained inside one does not.[1]

7 Objections

Memory is a sixth primitive. Memory is prominent in agent architectures (Sumers et al., 2024; Packer et al., 2023). We place it as a store in the world that Context reads and Output writes, governed by the persistence concern. The model has no memory of its own, and every implementation we examined (memory files loaded into the instructions, stores searched through tools, summaries carried forward, transcripts replayed on resume) is a decision about what Context includes and which actions Output offers. Making memory a primitive would split one decision, what the model is shown, across two components.

Planning is a sixth primitive. Planning is either something the model does in its own tokens, which needs no harness support, or a structure the harness imposes: a mode that restricts actions, a plan kept in a file, a separate planning call. Each of these is Control flow (which steps run, under what restrictions) or Context (where the plan is kept and shown). Every planning feature in §5 decomposed this way.

[1] In the companion book, a 0.6-billion-parameter model instruction-tuned on sessions recorded in its own harness rose from 2 to 18 of 32 held-out tasks in that harness, where the same base model post-trained by its developers for general use scored 14. The experiment, its data and its checks are described there (Anonymous, 2026); this paper makes no claim that rests on it.

A workflow is not a harness. A workflow fixes the sequence of model calls in code; an agent lets the model direct its own process (Schluntz & Zhang, 2024). Both are harnesses. They differ in CONTROL FLOW, and only there: a graph with named nodes and edges is one design decision, an open loop is another. The LangGraph deconstruction in §5 is included for this reason.

Multi-agent systems are something else. A system of several agents, each a model in a harness, is itself a harness whose OUTPUT includes delegating to another agent and whose INPUT includes that agent's result. Sub-agents, hand-offs and orchestrator–worker patterns are CONTROL FLOW decisions at the outer level. The primitives compose: a harness of harnesses still has five.

The harness will be absorbed into the model. Models increasingly learn behaviours that harnesses once imposed: when to stop, how to call tools in a trained format, when to reason before acting. But a model cannot open a file, keep a log or refuse a command on its own. Training moves *policy* into the model (what to do); the *mechanism* (how anything happens) remains in the harness. If models are trained inside a particular harness (§6), the harness defines the world the model was trained for, and its decomposition matters more, not less.

Five boxes can describe anything, and so explain nothing. The primitives are a coordinate system, not an explanation. Their value is in placing a design decision, a failure or a difference between two systems in one location, so that it can be studied, compared and changed independently of the rest. The claim is falsifiable in a specific way (§8), and the content is in the design decisions of §3 and the deconstruction of §5, not in the boxes.

8 Limitations and Threats to Validity

What would refute the claim. The position fails if a component of a working harness is found that (i) satisfies none of the five definitions, (ii) is not the answer to one of the six concerns within a primitive, and (iii) is not a nested harness. We invite reports of such a component. Three classes seemed most likely to produce one before the deconstruction: learned components inside the harness, search or decoding loops, and control flow generated by the model at run time. The first and third arose in the six harnesses and were placed by the rules of §4.1; search and decoding loops did not arise and remain untested.

Selection. Six harnesses, five of them coding agents and one a workflow framework used for a coding example. Harnesses for other domains (browsing, voice, robotics, customer service) were not examined, and the design spaces in §3 are drawn from what these six do. The selection criterion was wide use and the availability of primary sources; it is not a random sample.

Documentation-only analysis. Claude Code's source is not published. Its deconstruction rests on its documentation and Anthropic's published descriptions, which state what the harness does but not always how. Negative definitions are harder to confirm from documentation than from source, and a documented feature may differ from the shipped one.

The harness is not always in one place. Two of the six run part of their harness behind the provider's API (§5.7). A deconstruction from client source or documentation describes the local part; the part on the provider's side is visible only through its effects. As providers absorb more harness functions, this gap will grow.

Products change. Every assignment is pinned to a commit or a documentation date. Several of the six harnesses change weekly. The deconstruction describes them on those dates, and a reader checking it later should use the pinned versions.

Who made the assignments. The assignments were made by the author with an AI coding agent as a research instrument (see the disclosure), against the definitions of §3 and the rules of §4.1, and each cites its

Table 3: The five primitives, the step of the agent cycle each serves, and the CoALA component it corresponds to.

Primitive	Mechanism	In the agent cycle	CoALA
Input	capture requests and results	perceive	observations
Context	assemble what the model is shown	working and long-term memory	working memory; retrieval
Model interface	request a response	decide	reasoning (by the model)
Output	show words, carry out actions	act	grounding actions; learning (writes)
Control flow	go again, hand back, or stop	the cycle	decision cycle

evidence. No second, independent decomposition was made, so the agreement between two people applying the definitions is unmeasured. The rules in §4.1 are where disagreement would most likely arise: they follow from the definitions, but another reader could draw some boundaries elsewhere.

Stipulated boundaries. Backup versus routing, receiving versus rendering, the reader of a record and the consumer of an answer are rules we had to state. Their value is in making each choice explicit and checkable, not in being the only possible choice.

No experimental test. The paper does not vary one primitive at a time and measure the effect on performance. Fan et al. (2026) do this for three decisions in a coding harness; a study across all five primitives would test whether the vocabulary captures the factors that matter, and is future work. The one quantitative result we cite (§6, footnote) comes from the companion book and is not part of this paper's argument.

The model is held fixed. Training a model on traces from a harness moves work between the two components. We note this in §6 and do not analyze the trade-off.

9 Related Work

Agents and language agents. In artificial intelligence, an agent perceives its environment and acts on it (Russell & Norvig, 2020), in a cycle supported by memory. The loop that harnesses implement goes back to ReAct (Yao et al., 2023), which interleaves a model's reasoning with actions whose results return to it as observations. Later patterns live in particular primitives: Reflexion (Shinn et al., 2023) writes reflections on failed attempts and carries them into the next, a decision in Control flow and Context; MemGPT (Packer et al., 2023) pages information in and out of a limited window, a decision in Context; Toolformer (Schick et al., 2023) trains the model to emit tool calls, moving policy into the model while the calls are still carried out by Output.

CoALA. Sumers et al. (2024) organize language agents by memory (working, episodic, semantic, procedural), an action space of external and internal actions, and a decision cycle. CoALA describes what an agent *is*, as a cognitive architecture; this paper describes what software an agent is *made of*. The correspondence is in Table 3. Two things in this paper have no counterpart in CoALA: Model interface as a component, and the production concerns as a level of description distinct from the architecture. CoALA's memory modules, with the retrieval and learning actions that read and write them, are design decisions in Context and Output here, not a component.

Agent-computer interfaces. Yang et al. (2024) show that the interface through which a model acts on a computer (the commands it is given, their output format, guardrails such as linting edits) changes performance on software tasks, and name it the agent-computer interface. In our terms the interface is a set of design decisions in Output and Context. The present paper asks what the complete set of components is, of which the interface is part.

Workflows and agents. Schluntz & Zhang (2024) distinguish workflows, in which code fixes the sequence of model calls, from agents, in which the model directs its own process, and describe patterns such as prompt chaining, routing and orchestrator–workers built on an "augmented LLM". We treat the distinction as a property of Control flow, and decompose the augmented LLM into Context (retrieval, memory) and Output (tools) around a Model interface.

The harness as a variable. Zhang et al. (2026) argue that harness configuration can account for more variance in performance than the choice of model, and ask that the harness be disclosed when agents are compared. Fan et al. (2026) hold a coding harness's loop fixed and vary planning, the action space and context management across four models, finding that each changes agent behaviour. Their three dimensions were chosen independently of this paper and each falls within one primitive: planning in Control flow, the action space in Output, context management in Context. The present paper supplies what both assume: a statement of what the harness's parts are, which a disclosure can follow and a study can vary. Jin et al. (2026) use "primitives" for recurring patterns of model calls in multi-agent systems (review, voting, planning and execution); in our terms these are shapes of Control flow, and the two uses of the word are complementary.

10 Conclusion

An LLM agent is a model wrapped in a harness, and a harness is five mechanistic primitives: Input, Context, Model interface, Output and Control flow. What distinguishes one harness from another is the design decisions made within those primitives, and what makes a harness fit to run is the answers it gives to six questions asked of each. Six harnesses in wide use, deconstructed from their sources, fit this description with nothing left over. The vocabulary gives an engineer five places to look before blaming the model, and gives a benchmark a way to say which harness produced a score.

Disclosure of AI use

AI tools were used in two roles.

As a research instrument. The deconstruction of §5 was carried out with an AI coding agent (Claude, Anthropic) under the author's direction. The agent read each harness's source at the pinned commit, or its documentation on the stated date, applied the definitions of §3 and the rules of §4.1, and recorded every assignment with a file-and-line or URL citation. The author reviewed the assignments and the hard cases. Every assignment in Appendix B can be checked against its cited source without the agent.

As a writing assistant. The author set the position, the structure, the definitions and the three-level vocabulary, and reviewed every section. An AI assistant drafted prose from the author's outline and the companion repository, and edited for consistency. The minimal harnesses in Appendix A and the transcripts they cite are the author's, recorded in the companion repository.

References

Anomaly and contributors. opencode. https://github.com/anomalyco/opencode, 2026. Commit 654b79b, 2026-10-15.

Anonymous. Building agents: companion book and repository, 2026. Anonymized for review. The minimal harnesses and transcripts it cites are in the supplementary material.

Anthropic. Raising the bar on SWE-bench verified with Claude 3.5 Sonnet. https://www.anthropic.com/research/swe-bench-sonnet, 2025.

Anthropic. Claude Code documentation. https://code.claude.com/docs/en/how-claude-code-works, 2026. Documentation as read on 2026-10-15.

Run-Ze Fan, Zihao Zhang, Simin Ma, Yebowen Hu, Shouju Wang, Kaiqiang Song, Fei Liu, Hamed Zamani, and Xiaoyang Wang. An empirical study of harness design for coding agents. *arXiv preprint arXiv:2609.20804*, 2026.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024. arXiv:2310.06770.

Haibo Jin, Peng Kuang, Ye Yu, Xiaopeng Yuan, and Haohan Wang. Agent primitives: Reusable latent building blocks for multi-agent systems. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. arXiv:2602.03695.

Sayash Kapoor et al. Holistic agent leaderboard, 2025.

LangChain. LangGraph. https://github.com/langchain-ai/langgraph, 2026. Commit 5eb60ae, 2026-10-14; documentation at https://docs.langchain.com/oss/python/langgraph/overview.

METR. An update on our preliminary evaluations of Claude 3.5 Sonnet and o1. https://metr.org/blog/2025-01-31-update-sonnet-o1-evals/, 2025.

METR. Measuring time horizon using Claude Code and Codex. https://evals.alignment.org/notes/2026-02-13-measuring-time-horizon-using-claude-code-and-codex/, 2026.

Model Context Protocol. Model Context Protocol: Architecture overview. https://modelcontextprotocol.io/docs/learn/architecture, 2026. Accessed 2026-10-15.

Nous Research. Hermes Agent. https://github.com/NousResearch/hermes-agent, 2026. Commit 7bf3766, 2026-10-15.

OpenAI. Codex CLI. https://github.com/openai/codex, 2026. Commit f0c5077, 2026-10-15.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023.

Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023. arXiv:2302.04761.

Erik Schluntz and Barry Zhang. Building effective agents. https://www.anthropic.com/engineering/building-effective-agents, 2024.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. arXiv:2303.11366.

Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024. arXiv:2309.02427.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *International Conference on Machine Learning*, 2024. arXiv:2402.01030.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, 2024. arXiv:2405.15793.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. arXiv:2210.03629.

J. Young. Effective harnesses for long-running agents. https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents, 2025.

Mario Zechner. pi: a minimal coding agent. https://github.com/badlogic/pi-mono, 2026. Commit 5ab7fe7, 2026-10-15.

Yunbei Zhang, Janet Wang, Yingqiang Ge, Weijie Xu, Jihun Hamm, and Chandan K. Reddy. Stop comparing LLM agents without disclosing the harness. *arXiv preprint arXiv:2605.23950*, 2026.

A The Minimal Harnesses, One Primitive at a Time

The definitions in §3 were drawn from five programs, each the previous one plus one primitive, all run on the same task: a two-file Python project whose `total()` subtracts a discount as a flat amount where the test expects a percentage, and the request *"The tests are failing. Find out why and fix it."* The model is Claude Sonnet through Anthropic's Messages API; the files are `harness/1_model_interface.py` to `harness/quark.py` in the companion repository (Anonymous, 2026), reproduced here without their dependency headers. The transcripts quoted are from the recorded runs there. In the transcripts, `>` is what the person typed, `<` what the model said, and `$` a command it ran.

A.1 Model interface alone

```
from anthropic import Anthropic

model = Anthropic()


def request_response(context):
    return model.messages.create(model="claude-sonnet-5-5", max_tokens=16384, **context)


response = request_response({"messages": [{"role": "user", "content": "The tests are failing. Find out why and fix it."}]})
print(response.model_dump_json(indent=2))
```

The model answers, but it can only answer: *"I don't have access to your code, test files, or test output. Nothing came through in your message besides the request."* The request is written into the program, so nobody can ask it anything else. What is missing is a way for something to come in.

A.2 Plus Input

```


def capture_input():
    return input("\n> ")
```

Anyone can now type the request, and the answer is the same: *"I don't have the code or the test output yet, so I can't tell why the tests are failing. Could you share…"* It can only talk. What is missing is a way to act.

A.3 Plus Output

```

model = Anthropic()

bash = {"name": "bash", "description": "Run a shell command",
        "input_schema": {"type": "object", "properties": {"command": {"type": "string"}}, "required": ["command"]}}


def request_response(context):
    return model.messages.create(model="claude-sonnet-5-5", max_tokens=16384, **context)


def capture_input():
    return input("\n> ")
```

```
def handle_output(response):
    tool_results = []
    for block in response.content:
        if block.type == "text":
            print("< " + block.text.replace("\n", "\n  "))
        if block.type == "tool_use":
```

A tool is described to the model, and `handle_output` runs each command the model asks for and collects the result:

```
> The tests are failing. Find out why and fix it.
< I'll start by exploring the project and running the tests.
$ pwd; ls -la; git status 2>/dev/null | head
```

Then it stops. The command ran and printed the files, and the result went nowhere: nothing carries it back to the model. What is missing is the decision to go again.

A.4 Plus Control flow

```
def control_flow():
    conversation = []
    while True:
        conversation.append({"role": "user", "content": capture_input()})
        while True:
            response = request_response({"tools": [bash], "messages": conversation})
            conversation.append({"role": "assistant", "content": response.content})
            tool_results = handle_output(response)
            if not tool_results:
```

With the results sent back and the model called again until it stops requesting actions, the same model, tool and request finish the task:

```
> The tests are failing. Find out why and fix it.
< Let me start by looking at the project and running the tests.
$ pwd; ls -la; git status 2>/dev/null | head
$ cat prices.py test_prices.py; python -m pytest -q 2>&1 | tail -30
$ python -m unittest -v 2>&1 | tail -20
< The docstring says the discount is a percentage, but the code subtracts it
  as a flat amount. So 25 - 10 = 15 instead of 25 x 0.9 = 22.5. I'll fix the
  code to match the documented behavior.
$ sed -i 's|return round(subtotal - discount, 2)|return round(subtotal * (1 - discount / 100), 2)|' prices.py && cat prices.py
    && python -m unittest 2>&1 | tail -5
< The tests pass now. The bug was in prices.py, not in the tests.
```

This is an agent. Asked a question about itself, though, it has to go looking:

```
> Who are you, and where are you working?
< I'll check the environment first so I can answer accurately.
$ echo "user: $(whoami)"; echo "host: $(hostname)"; echo "cwd: $(pwd)"; uname -a; ls -la | head -20
< I'm an AI assistant. I can run shell commands, and I use that to read files,
  run code and make changes in the environment I'm given.
```

It does not know its name, its body, its working directory or the date. What is missing is what the model is shown.

A.5 Plus Context: the complete harness

```
import subprocess, os, datetime
from anthropic import Anthropic

model = Anthropic()
bash = {"name": "bash", "description": "Run a shell command",
        "input_schema": {"type": "object", "properties": {"command": {"type": "string"}}, "required": ["command"]}}


def capture_input():
    return input("\n> ")
```

```
def assemble_context(conversation):
    instructions = f"You are quark, an agent. You act through bash, in {os.getcwd()}. Today is {datetime.date.today()}."
    return {"system": instructions, "tools": [bash], "messages": conversation}


def request_response(context):
    return model.messages.create(model="claude-sonnet-5-5", max_tokens=16384, **context)


def handle_output(response):
    tool_results = []
    for block in response.content:
        if block.type == "text":
            print("< " + block.text.replace("\n", "\n  "))
        if block.type == "tool_use":
            print(f"$ {block.input['command']}")
            ran = subprocess.run(block.input["command"], shell=True, capture_output=True, text=True)
            tool_results.append({"type": "tool_result", "tool_use_id": block.id, "content": ran.stdout + ran.stderr or "(no
     output)"})
    return tool_results


def control_flow():
    conversation = []
    while True:
        conversation.append({"role": "user", "content": capture_input()})
        while True:
            response = request_response(assemble_context(conversation))
            conversation.append({"role": "assistant", "content": response.content})
            tool_results = handle_output(response)
            if not tool_results:
                break                                   # done: hand back to the person
            conversation.append({"role": "user", "content": tool_results})   # a tool ran: go again


control_flow()
```

One sentence of context, assembled on every call, answers the question without any action:

```
> Who are you, and where are you working?
< I'm quark, an agent that works by running shell commands through bash. My
  working directory is /tmp/shop. Today's date is 2026-10-07.
```

This is the whole harness: fifty lines, five functions, one per primitive. It runs commands on the host without asking and forgets everything when it exits; the production concerns of §4 are what it lacks, and the companion repository adds them one question at a time to produce `quark_production.py`. The same repository later replaces `request_response` with a function that runs a local model and translates between the Messages format and the model's chat template, leaving the other four primitives unchanged; that is the porting claim of §6 in practice.

B Evidence for the Deconstruction

Every assignment in §5, with its evidence, is given in the supplementary material: a 50-page PDF with the same section structure as this appendix, and the source files it was generated from, one per harness. For harnesses with published source, a citation is a path and a line range at the pinned commit; for Claude Code it is a documentation page and section heading on the date read.